Deploying the platform for free

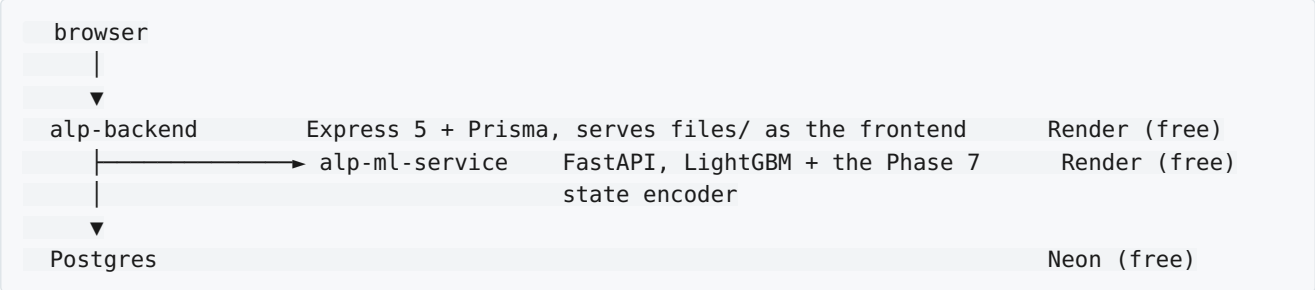
A recipe for putting the running platform behind a public URL, so someone can click a link and use it without installing anything.

It covers the platform only — the backend, the frontend and the model service. The research outputs (figures, benchmark JSON, the two PDFs, the archival corpora) are deliberately left out of the deployed repository.

Everything here is free. Two accounts are needed: **Neon** for the database and **Render** for the two services. Both accept a GitHub login.

Expect **40–60 minutes** the first time, most of it spent watching Render build the images.

What you are deploying



The frontend needs no configuration: `files/script.js` resolves the API base from `window.location.origin`, so it talks to whatever host serves it.

Before you start — three things worth knowing

Free instances sleep. Render spins a free service down after 15 minutes of inactivity, and the next request wakes it. A cold start is roughly 50 seconds here, because the model service loads torch and the state encoder on boot. The first person to open the link after a quiet period sees a blank page for that long. Open the link yourself a minute before anyone else does and it is warm.

Free instances share 750 hours a month across all of your free services. Two services that sleep when idle stay well inside that. Two services kept awake by a pinger do not — don't add one.

Use Neon, not Render's database. Render's free Postgres is deleted 30 days after it is created. Neon's free tier does not expire.

Step 1 — Make the deploy copy

On the machine that has the project. This produces a folder holding only what the platform needs, so no figure, report or PDF ever reaches GitHub.

Order matters in the block below: rsync applies the first rule that matches, so every `--include` has to come before the `--exclude` that would otherwise swallow it.

```
cd ~/Documents                  # wherever the project sits
rsync -a --delete \
  --exclude '.git' \
  --exclude 'node_modules' \
  --exclude '__pycache__' \
  --exclude '.pytest_cache' \
  --exclude '.venv' \
  --exclude 'venv' \
  --exclude 'mlruns' \
  --exclude '.env' \
  --exclude '*.pdf' \
  --exclude 'docs' \
  --exclude 'artifacts-legacy-invalid' \
  --include 'data/' \
  --include 'data/items/***' \
  --exclude 'data/*' \
  --include 'artifacts/' \
  --include 'artifacts/models/***' \
  --include 'artifacts/evaluation/' \
  --include 'artifacts/evaluation/decision-explanations.json' \
  --include 'artifacts/evaluation/validation-gate.json' \
  --include 'artifacts/datasets/' \
  --include 'artifacts/datasets/item-parameters-v1.csv' \
  --exclude 'artifacts/evaluation/*' \
  --exclude 'artifacts/datasets/*' \
  --exclude 'artifacts/*' \
  alp-system/ alp-deploy/
```

Check what survived:

```
du -sh alp-deploy                                  # expect roughly 20 MB
find alp-deploy -name '*.pdf' | wc -l              # must print 0
ls alp-deploy/artifacts/figures 2>/dev/null        # must say "No such file"
ls alp-deploy/artifacts/models/state-encoders-by-rung.pt \
   alp-deploy/artifacts/evaluation/validation-gate.json \
   alp-deploy/artifacts/datasets/item-parameters-v1.csv \
   alp-deploy/data/items/item-bank-v1.json         # all four must exist
```

> **Why those four have to stay.** They are what the model service opens while answering a request, and a Render service has no mounted volume, so anything read at request time must be inside the image.
>
> - `artifacts/models/` (17 MB) — the trained LightGBM artifact, the Phase 7 state encoders and calibrators, the BKT parameters, and `item-response-stats.json`. That last one is why the 1.3 GB corpus is not needed: the statistics are cached there.
> - `artifacts/evaluation/validation-gate.json` (6 KB) — Phase 7's gate report.
> - `artifacts/datasets/item-parameters-v1.csv` (6 KB) — the Rasch parameters the rule arms band on.
> - `data/items/item-bank-v1.json` (160 KB) — the item bank.
>
> These are weights and configuration, not results. No figure, no benchmark number and neither PDF is among them.

Copy `alp-deploy/` to the personal laptop (zip it, AirDrop it, whatever).

Step 2 — Push it to GitHub as a fresh private repository

On the personal laptop. A fresh `git init` matters: the original repository's history contains the figures and the PDFs, and pushing that history would carry them to GitHub even though the current commit no longer has them.

```
cd ~/alp-deploy
git init
git add -A
git commit -m "Adaptive learning platform: deployable subset"
```

Then create the repository. With the GitHub CLI:

```
gh repo create alp-platform --private --source=. --push
```

Without it: create an empty private repo named `alp-platform` at https://github.com/new, then

```
git remote add origin https://github.com/<your-username>/alp-platform.git
git branch -M main
git push -u origin main
```

Before moving on, open the repo on github.com and confirm there is no `artifacts/figures` directory and no `docs` directory.

Step 3 — Create the database on Neon

1. Sign up at https://neon.tech with GitHub.
2. **Create project** — name it `alp`, take the default Postgres version, and pick the region closest to Oregon (Render's free region), `AWS us-west-2`.
3. On the project dashboard, open **Connection string** and copy the **pooled** connection string. It looks like:

`postgresql://alp_owner:XXXXXXXX@ep-something-pooler.us-west-2.aws.neon.tech/alp?sslmode=require`

Keep that string somewhere for the next step. `sslmode=require` must stay on the end — Prisma will not connect to Neon without it.

Step 4 — Deploy both services on Render

1. Sign up at https://render.com with GitHub and give it access to the `alp-platform` repository.
2. **New → Blueprint**, pick `alp-platform`. Render finds `render.yaml` and offers to create `alp-ml-service` and `alp-backend`.
3. It will prompt for the two values marked `sync: false`. Fill in what you can now: - `DATABASE_URL` → the Neon string from step 3. - `ML_SERVICE_URL` → leave it blank or put `http://placeholder`; the real value does not exist yet. Step 5 fixes it.
4. **Apply**. Both services start building.

The model service takes **10–15 minutes** on its first build — it is downloading the CPU build of torch. The backend takes about 3 minutes. Watch the logs from each service's page.

`alp-ml-service` is up when its log ends with `Uvicorn running on http://0.0.0.0:10000` and the service shows **Live**.

Step 5 — Point the backend at the model service

This is the one wiring step the blueprint cannot do for you, because the URL is only assigned once the service exists.

1. Open **alp-ml-service** in Render. Copy its URL from the top of the page — something like `https://alp-ml-service.onrender.com`.
2. Open **alp-backend → Environment**.
3. Set `ML_SERVICE_URL` to that URL. **No trailing slash.**
4. **Save changes.** The backend redeploys, about a minute.

Step 6 — Check it, then send the link

Open `https://alp-backend.onrender.com/health` — this also wakes the service, so give it a minute the first time.

```
{"status":"ok","database":"up"}
```

And the model service, `https://alp-ml-service.onrender.com/health`:

```
{"status":"ok","model_loaded":true,"model_name":"study1-assistments_2009-
lightgbm","model_source":"registry"}
```

`model_loaded: true` is the one to check. If it says `false`, the model artifacts did not make it into the image — go back to step 1 and confirm `artifacts/models` is in the repository on GitHub.

Then one more, because the failure it catches is silent. Paste this into a terminal, substituting your ml-service URL:

```
curl -s -X POST https://alp-ml-service.onrender.com/v1/state \
  -H 'Content-Type: application/json' \
  -d '{"session_id":"smoke","attempts":
[{"session_id":"smoke","item_id":"I001","concept_id":"C01","correct":true,"difficulty_score":5
,"response_time_ms":4200,"item_b":-0.3,"position":0,"question_number":0}]}'
```

The reply must contain `"admitted":["knowledge"]` and a `knowledge` number under `states`:

```
{"states":{"knowledge":0.5765765905380249}, ... "gate":{"admitted":["knowledge"], ...}}
```

If `states` is `{}` and `admitted` is `[]`, the service is running but `validation-gate.json` is missing from the image. Everything still returns HTTP 200 and the app still works — it just quietly stops using the estimated state and falls back to a fixed 0.5, which is the one failure here that looks like success. Fix it in step 1, not by restarting.

Then open `https://alp-backend.onrender.com` in a browser and run through a session yourself before sending it on. The question bank is seeded automatically on every boot, so there is nothing to load by hand.

The link to send is the backend one: `https://alp-backend.onrender.com`.

Tell her it may take up to a minute to load the first time — otherwise a cold start reads as a broken link.

If something goes wrong

The page loads but every action fails. `ML_SERVICE_URL` is wrong, or the model service is asleep and the backend timed out waiting for it. Open the ml-service URL directly to wake it, then retry. Check for a trailing slash.

Backend log: `Can't reach database server`. The Neon string is wrong or lost its `?sslmode=require`. Copy the pooled string again from Neon.

Backend log: `P3009` or a failed migration. The database has a partial schema. In Neon, delete the `public` schema's tables (or just delete the project and make a new one), update `DATABASE_URL`, redeploy.

ml-service log: `Killed`, or the deploy loops. Out of memory — 512 MB is not much with torch in it. Lower `ALP_LIVE_SLOTS` from `8` to `4` in the service's environment.

ml-service build fails on torch. The pin in `requirements-serve.txt` has no CPU wheel for that Python. Change `torch==2.13.0` to plain `torch`, commit, push; Render redeploys on push.

`/health` says `model_loaded: false`. `artifacts/models` is missing from the repo — the most likely cause is an over-broad exclude in step 1.

Cost, and what happens later

Nothing here bills. Neon's free project stays; Render's free services stay. Both idle down and wake on request.

If the sleeping becomes the problem — she is dipping into it over several days and keeps meeting a cold start — Render's Starter plan at $7/month per service removes the spin-down. Only the backend really needs it; the model service waking a few seconds later is not noticeable once the page is already up.

To take it down: delete the two services in Render and the project in Neon. Deleting the GitHub repository is separate, and worth doing when the demo is over.